AMRITA VISHWA VIDYAPEETHAM

Department: AIE

FINAL PROJECT REPORT

INTELLIGENT EMERGENCY RESOURCE ALLOCATION & AMBULANCE DISPATCH SYSTEM

Using Point Location, Orthogonal Range Search and Fractional Cascading — Implemented in Java

Submitted by

S. No.	Student Name	Register Number
1	SHARATH SACHIN S	CH.SC.U4AIE25071
2	NITESH RAM RE	CH.SC.U4AIE25075
3	SIVAGURU R M	CH.SC.U4AIE25044
4	RAHUL BHARADWAJ	CH.SC.U4AIE25039

Course Name: DATA STRUCTURES AND ALGORITHMS 2
Course Code: 22AIE203
Department: AIE
Institution: AMRITA VISHWA VIDYAPEETHAM
Academic Year: 2026–27
Faculty / Project Guide: BHARATHI MOHAN

TABLE OF CONTENTS

1. INTRODUCTION **1**

2. DOMAIN OVERVIEW **1**

2.1 Emergency Dispatch as a Spatial Problem 1

2.2 Computational Geometry and Spatial Indexing 2

3. EXISTING SYSTEM AND LIMITATIONS **2**

3.1 Existing System 2

3.2 Limitations 2

4. PROBLEM FORMULATION **3**

4.1 Input 3

4.2 Operations 4

4.3 Processing 4

4.4 Output 4

5. PROBLEM STATEMENT **4**

6. OBJECTIVES **5**

7. SELECTED DATA STRUCTURES AND ALGORITHMS **5**

7.1 Orthogonal Range Tree 5

7.2 Fractional Cascading 6

7.3 Point Location 6

7.4 Nearest-Neighbour Search 7

7.5 Radius Search and Filtering 7

7.6 Severity-Based Allocation 7

8. JUSTIFICATION FOR DATA STRUCTURE AND ALGORITHM SELECTION **8**

9. METHODOLOGY **9**

9.1 System Architecture 9

9.2 Data Flow 9

9.3 Workflow 9

10. ALGORITHM DESIGN AND PSEUDOCODE **10**

10.1 Algorithm 1 — Building the Orthogonal Range Tree 11

10.2 Algorithm 2 — Orthogonal Range Search 11

10.3 Algorithm 3 — Fractional Cascading Search 12

10.4 Algorithm 4 — Point Location Using Slab Decomposition 12

10.5 Algorithm 5 — Nearest Available Resource 13

10.6 Algorithm 6 — Hospitals with ICU Within Radius 13

10.7 Algorithm 7 — Severity-Based Allocation 13

10.8 Algorithm 8 — Ambulance Redistribution 14

11. IMPLEMENTATION **14**

11.1 Programming Language 14

11.2 Development Environment 14

11.3 Project Structure 14

11.4 Important Classes 15

11.5 Data Files 15

11.6 Implementation Details 15

12. COMPLEXITY ANALYSIS **16**

13. EXPERIMENTAL SETUP AND TEST CASES **16**

13.1 Experimental Environment 16

13.2 Dataset 17
13.3 Test Cases 17
14. RESULTS AND DISCUSSION 17
14.1 Functional Results 17
14.2 Complexity Results 18
14.3 Correctness Results 19
14.4 Discussion 19
15. CONCLUSION AND FUTURE SCOPE 19
15.1 Conclusion 19
15.2 Future Scope 19
REFERENCES 20
APPENDIX 22
A.1 Project Directory Structure 22
A.2 Build and Run Commands 22
A.3 Selected Source Code 22
A.4 Complete Demonstration Output 27
A.5 Complete Verification Output 28
A.6 Additional Pseudocode 29
A.7 Additional Test Cases from the Demonstration 29
A.8 Sample-City Dataset 29

1. INTRODUCTION

When an emergency call reports a road accident, a cardiac arrest or a fire, the control room must decide quickly which units to send and where the patient should go. In this project the decision involves ambulances, fire engines, police units and hospitals with intensive-care (ICU) beds, and every such decision is spatial: it depends on where the incident is and where each resource is at that moment. The dispatcher's questions are geometric — *which ambulances are free in this district, which free ambulance is nearest, which hospitals within 5 km still have an ICU bed, and which sector owns this incident?* — and the project materials describe them as being asked repeatedly, for every call, over resources whose availability changes after each dispatch.

The straightforward implementation keeps the resources in a list and examines every one of them for every question. This costs $\Theta(n)$ time per query for n resources regardless of how small the answer is, the cost grows linearly with the fleet, and it multiplies inside a single allocation: a critical road accident in this system triggers up to five separate nearest-unit searches (two ambulances, an ICU hospital, a fire engine and a police unit). Because the time spent computing a decision adds to the response time, the computation itself should not become the slow step.

The **Intelligent Emergency Resource Allocation & Ambulance Dispatch System** replaces repeated scanning with preprocessing. Resource positions are organised once into a two-dimensional **orthogonal range tree augmented with fractional cascading** (`RangeTree2D`), which reports all resources inside an axis-aligned rectangle in $O(\log n + k)$ time, where k is the number of points reported. The rectangular city sectors are stored in a **slab-decomposition point-location** structure (`PointLocation`) that finds the sector containing an incident in logarithmic time. On top of these, `DispatchSystem` implements the six operational queries — available ambulances in a region, nearest available ambulance, ICU hospitals within a radius, severity-based allocation, ambulance redistribution and a coverage report — together with incident-sector identification. Data structures are therefore the core of the project: the $O(n \log n)$ build is amortised over all later queries, fractional cascading removes repeated binary searches, and the brute-force scan is kept only as a test oracle against which every structure is verified.

2. DOMAIN OVERVIEW

2.1 Emergency Dispatch as a Spatial Problem

The project belongs to the domain of emergency resource allocation and dispatch. A dispatch centre tracks *mobile units* — ambulances, fire engines and police units — whose availability changes when they are dispatched or return to service, and *fixed facilities* — hospitals — whose relevant state here is the number of free ICU beds. Each incident has a location, a type and a severity. The project represents every resource as a point in the plane carrying its attributes as a payload, which turns dispatch questions into geometric queries (Table 1).

Table 1: Emergency resources modelled in the project

Resource type	State used	Role in dispatch
AMBULANCE	available flag	Allocated by severity; relocated during redistribution
FIRE_TRUCK	available flag	Added for road accidents, fires and natural disasters
POLICE_UNIT	available flag	Added for road accidents, fires, natural disasters and crime
HOSPITAL	free ICU beds	Receives CRITICAL and HIGH incidents; one ICU bed reserved each

Four query primitives recur: *region* queries (all units in a district), *proximity* queries (the nearest unit of a type), *radius queries with an attribute filter* (hospitals within a distance that have ICU capacity) and *containment* queries (the sector containing an incident). The workload is dominated by repetition — one allocation issues several queries and the coverage report issues three per sector — and the data are uneven and changing: a sector can lose all its free ambulances after a few calls, as the project demonstrates for the South-East sector, so units must be moved and the stored positions updated.

2.2 Computational Geometry and Spatial Indexing

Orthogonal range searching, planar point location and nearest-neighbour search are classical problems of computational geometry [1], [12]. A *spatial index* organises points so that a query touches only the data relevant to it. The dispatch model suits this approach because queries are far more frequent than changes of position, because query regions are rectangles or can be bounded by them (a circle of radius r lies inside a square of half-side r), and because the sectors are rectangles, which makes slab-based point location sufficient. Balanced search trees, sorted arrays, merging, binary search and cross-linking pointers are the DSA tools combined to obtain logarithmic query costs.

3. EXISTING SYSTEM AND LIMITATIONS

3.1 Existing System

The baseline is the linear scan: resources are kept in an unordered list and every query walks the whole list — testing $x_1 \le x \le x_2$ and $y_1 \le y \le y_2$ for a region, computing every Euclidean distance for a nearest or radius query, and testing every zone rectangle for sector identification. Nothing computed for one query is reused by the next. The project contains this baseline in concrete form: the verification suite implements these scans (`bruteRange`, `bruteNearest` and a brute-force circle test) as reference answers.

```
LINEAR-RANGE(S, [x1,x2] × [y1,y2], keep)
    H ← ∅
    for each resource s in S do
        if x1 ≤ s.x ≤ x2 and y1 ≤ s.y ≤ y2
           and keep(s) then H ← H ∪ {s}
    return H
```

```
LINEAR-NEAREST(S, q, keep)
     best ← NIL ;  d* ← ∞
     for each resource s in S do
         if keep(s) and dist(s, q) < d* then
             best ← s ;  d* ← dist(s, q)
     return best
```

3.2 Limitations

Every query costs $\Theta(n)$ regardless of the size of its answer, so Q queries cost $\Theta(Qn)$ and doubling the fleet doubles the work per query. The cost compounds within one dispatch — a CRITICAL road accident needs about 5n distance computations — and a coverage report over z sectors costs $\Theta(zn)$. The scan has no notion of locality and cannot exploit the fact that the same resource set is queried again and again. A preprocessing approach is needed: pay a

one-time construction cost, then answer each query in time close to the size of its output. Table 2 compares the two approaches; the bounds are derived in Section 12.

Table 2: Baseline versus proposed spatial index (n resources, k reported points, z sectors)

Operation	Linear scan	Proposed spatial index	Structure used
Preprocessing	none	O(n log n)	RangeTree2D + PointLocation
Rectangular region report	Θ(n)	O(log n + k)	Range tree + fractional cascading
Nearest available unit	Θ(n)	O(log n + k) per window	Expanding-window range queries
ICU hospitals within radius	Θ(n)	O(log n + k) for bounding square	Range query + exact filter
Sector of an incident	Θ(z)	O(log z)	Slab decomposition
Mark unit busy / free	O(1)	O(1), no rebuild	Mutable availability flag
Relocate a unit	O(1)	O(n log n) rebuild	Full reconstruction

4. PROBLEM FORMULATION

4.1 Input

Coordinate model. Locations lie on a planar kilometre grid (x east–west, y north–south), which the source code describes as a local projection of latitude and longitude, so Euclidean distance $d(p, q) = \sqrt{((p_x - q_x)^2 + (p_y - q_y)^2)}$ approximates straight-line ground distance in km. The sample city is a 30 × 30 km square with four sectors and sixteen resources (Figure 1).

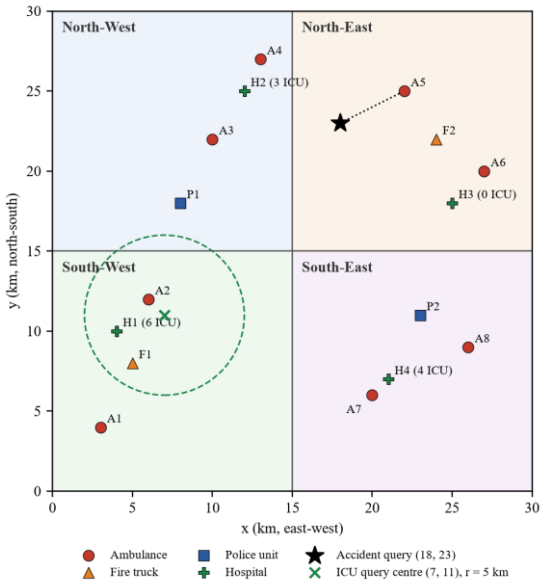


Figure 1: Sample 30 × 30 km city from data/resources.csv and data/zones.csv, with the demonstration accident at (18, 23) and the 5 km ICU search around (7, 11)

Table 3: Input entities of the computational model

Entity	Attributes	Source
Resource $s \in S$, $\|S\| = n$	id, name, type ∈ {AMBULANCE, FIRE_TRUCK, POLICE_UNIT, HOSPITAL}, (x, y), availability, free ICU beds	resources.csv → EmergencyResource
Zone Z_i, $i = 1 \dots z$	name and non-overlapping rectangle [minX, maxX] × [minY, maxY]	zones.csv → Zone
Incident I	id, (x, y), type ∈ {ROAD_ACCIDENT, CARDIAC_ARREST, FIRE, NATURAL_DISASTER, MEDICAL, CRIME}, severity ∈ {CRITICAL, HIGH, MODERATE, LOW}	Incident (demo or console)
Query parameters	rectangle R, point q, radius ρ	DispatchSystem arguments

4.2 Operations

With $A(R) = \{s \in S : s.type = AMBULANCE, s.available, (s.x, s.y) \in R\}$ denoting the available ambulances in rectangle R, the supported operations are listed in Table 4.

Table 4: Operations supported by the system

#	Operation	Definition	Result
1	Available ambulances in a region	A(R) for a query rectangle R	List
2	Nearest available ambulance	argmin of d(s, q) over available ambulances s	One unit or none
3	Hospitals with ICU within radius	Hospitals with icuBeds > 0 and $d(s, q) \le \rho$, closest first (ρ = 5 km in the demo)	Ordered list
4	Severity-based allocation	Units assigned by severity and incident type (Section 7.6)	DispatchPlan
5	Ambulance redistribution	Each zone with $A(Z_i) = \emptyset$ receives the nearest surplus ambulance at its centre	Move log
6	Coverage report	Per zone: $\|A(Z_i)\|$, ICU within a radius of the centre, nearest ambulance	Table
7	Sector identification	The zone Z_i that contains q	Zone or none

4.3 Processing

In the *preprocessing* phase the range tree is built over all n resources of every type — availability is not part of the index — and the slab structure over the z zones. In the *query* phase each operation is reduced to orthogonal range reporting followed by a filter predicate (type, availability, ICU capacity or exact distance), or to point location. In the *update* phase dispatch, return to service and ICU reservation change attributes in place; only relocation, which changes a coordinate, rebuilds the index.

4.4 Output

The outputs are resource lists (region and radius queries), a single resource or none (nearest queries), the zone of an incident or *outside mapped sectors*, a `DispatchPlan` (zone, ambulances, ICU hospital, fire truck, police unit and any unmet requirements), a per-sector coverage table that flags sectors without a free ambulance, and a log of redistribution moves.

5. PROBLEM STATEMENT

Given a set S of n emergency resources on a planar kilometre grid, each with a type and a mutable availability state (and, for hospitals, a count of free ICU beds), and a set of z non-overlapping rectangular response sectors, design and implement data structures and

algorithms that repeatedly answer, for arbitrary incident locations, the six operational queries of Table 4 and the sector containing the incident. A linear scan costs O(n) per query and O(Qn) for Q queries, so the latency of the dispatch computation grows with both the fleet and the number of queries per incident. The problem is to replace the scan with a spatial index built once in O(n log n) time that reports rectangular ranges in O(log n + k) time and locates sectors in logarithmic time, while returning exactly the brute-force answers. The system is a computational model that uses straight-line distances on a 30 × 30 km sample city; it is not a production emergency platform.

6. OBJECTIVES

O1. Answer rectangular region queries in O(log n + k) time instead of by an O(n) scan.

O2. Implement a two-dimensional **orthogonal range tree** from first principles in Java.

O3. Add **fractional cascading** so that each query performs a single binary search.

O4. Implement **slab-decomposition point location** to identify the sector of an incident in O(log z).

O5. Build exact **nearest-resource** and **radius/ICU** queries on the same index.

O6. Encode a **severity-based dispatch policy**, maintain resource state and support **dynamic redistribution** with index rebuilding.

O7. **Analyse** the time and space complexity of every operation against the linear-scan baseline.

O8. **Validate correctness** against a brute-force reference, including edge cases.

7. SELECTED DATA STRUCTURES AND ALGORITHMS

7.1 Orthogonal Range Tree

An *orthogonal range query* reports the points of a set P inside an axis-aligned rectangle $[x_1, x_2] \times [y_1, y_2]$. The two-dimensional range tree [1, Ch. 5], [3], [4] answers it with two levels, implemented as `RangeTree2D<T>`. The **primary tree** is a balanced BST on x with the points in its leaves: the points are sorted by x once and split recursively at the median position, each internal node storing `splitX`, the largest x in its left subtree ($x \le$ splitX goes left), so the height is $\lceil \log_2 n \rceil$. The **associated structure** of every node is the array `ys` of the points in its subtree sorted by y.

A query descends to the **split node**, where the search paths for x_1 and x_2 diverge. Below it, whenever the path towards x_1 turns left, the right child's whole subtree lies inside $[x_1, x_2]$ and is a **canonical node**; the path towards x_2 is symmetric. The O(log n) canonical nodes, plus the two boundary leaves checked individually, cover exactly the points with x in $[x_1, x_2]$, and in each canonical node the points with y in $[y_1, y_2]$ are a contiguous run of `ys`, found by binary search for y_1 and scanned while $y \le y_2$. The build sorts once and merges n points per level, giving O(n log n) time and space; the plain query costs **$O(\log^2 n + k)$**. In the project this tree is the single spatial index: all sixteen resources are stored in it, and every region, nearest and radius query goes through it.

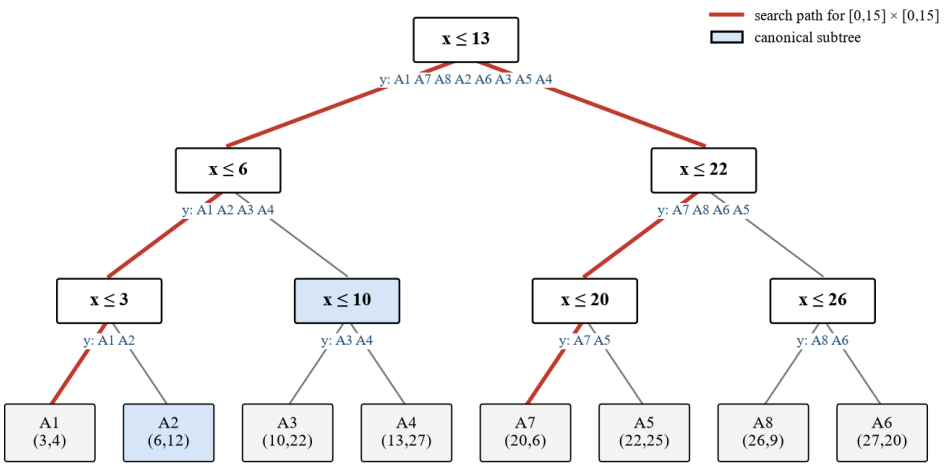


Figure 2: Range tree over the eight sample ambulances (as in the project presentation): split keys, y-arrays and the search for the South-West rectangle [0, 15] × [0, 15]

In Figure 2 the root ($x \le 13$) is the split node because $0 \le 13 \le 15$. The left path turns left at $x \le 6$ and $x \le 3$, making subtree $x \le 10$ and leaf A2 canonical: $x \le 10$ reports nothing (its first entry, A3, has $y = 22 > 15$), A2 ($y = 12$) is reported and the boundary leaf A1 (3, 4) passes its test. The right path ends at A7 (20, 6), which fails $x \le 15$. The result {A2, A1} matches the demonstration output.

7.2 Fractional Cascading

The extra logarithmic factor comes from repeating the *same* binary search for y_1 in every canonical node. Fractional cascading [2] removes it: a child's y-array is a subsequence of its parent's (the parent's is their merge), so the position of y_1 in each child can be stored in advance. For each index i of `ys`, the implementation stores `leftPtr[i]` and `rightPtr[i]`, the first index in the left and right child's arrays whose y is $\ge$ ys[i].y, plus a sentinel entry for thresholds above every stored y. If i is the first parent index with $y \ge y_1$, no parent entry — and hence no child entry — lies in $[y_1$, ys[i].y), so the stored pointer is also the first child entry with $y \ge y_1$. One binary search at the root followed by one pointer step per node gives **O(log n + k)**; the pointers are filled in during the merge in linear time, so the build stays O(n log n). This layered range tree [1] is how `RangeTree2D` describes itself.

Table 5: Root y-array and cascade pointers of Figure 2 (left child A1, A2, A3, A4; right child A7, A8, A6, A5)

Index i	0	1	2	3	4	5	6	7	8
root.ys[i] (y)	A1 (4)	A7 (6)	A8 (9)	A2 (12)	A6 (20)	A3 (22)	A5 (25)	A4 (27)	sentinel
leftPtr[i]	0	1	1	1	2	2	3	3	4
rightPtr[i]	0	0	1	2	2	3	3	4	4

For $y_1 = 10$ the root search returns i = 3 (A2, y = 12); `leftPtr[3]` = 1 selects A2 in the left child and `rightPtr[3]` = 2 selects A6 (y = 20) in the right child — both correct first entries with $y \ge 10$, found without further searching.

7.3 Point Location

Non-overlapping regions form a *planar subdivision*; point location reports the region containing a query point [1, Ch. 6]. `PointLocation<Z>` uses **slab decomposition** [7]: the distinct x-values of all vertical rectangle edges bound vertical **slabs**; no edge lies inside a

slab, so the rectangles spanning it form disjoint y-intervals, stored sorted by lower edge. A query binary-searches x among the slab boundaries, then y among the slab's lower edges, and checks the upper edge: **O(log z)**. Construction tests each slab's midpoint against every rectangle, $O(z^2 \log z)$ time and $O(z^2)$ space in the worst case. Both searches use an upper-bound rule, so a point on a shared boundary belongs to the sector to its right or above. The sample zones give boundaries {0, 15, 30} and two slabs (Table 6). `DispatchSystem.zoneOf` labels each dispatch plan with its sector — E1 at (19, 24) is routed to North-East — and finds donors' home sectors during redistribution.

Table 6: Slab structure built from data/zones.csv

Slab	x-range (km)	Sorted y-intervals → zone
0	[0, 15]	[0, 15] → South-West; [15, 30] → North-West
1	[15, 30]	[0, 15] → South-East; [15, 30] → North-East

7.4 Nearest-Neighbour Search

`RangeTree2D.nearest` builds an exact search from range reporting with an **expanding square window**. Starting with half-side r = 1 km around q, it applies the predicate (e.g. *available AMBULANCE*) to the reported points and doubles r until a point survives or r reaches R, the largest distance from q to a side of the data's bounding box. With candidate distance d = the minimum over the survivors, a final query with half-side d returns the exact nearest point: any point closer than d satisfies $|x - q_x| < d$ and $|y - q_y| < d$, so it lies in that square, even if the first window missed it. For the accident at (18, 23), the windows r = 1 and r = 2 are empty, r = 4 ([14, 22] × [19, 27]) contains A5, d = $\sqrt{20} \approx 4.47$ km, and the final window confirms A5.

7.5 Radius Search and Filtering

A circle of radius ρ lies inside its bounding square. `withinRadius` performs one range query on the square and keeps points with $d \leq \rho$ that satisfy the predicate; `hospitalsWithIcuWithinRadius` uses *type = HOSPITAL and icuBeds > 0* and sorts by distance. Around (7, 11) with ρ = 5 km, the square [2, 12] × [6, 16] reports F1, A2 and H1, and only H1 (City-General, 6 ICU beds, 3.16 km) survives. The square exceeds the circle by a factor of $4/\pi \approx 1.27$, so extra candidates come only from its corners.

7.6 Severity-Based Allocation

The policy is stored as data in the `Severity` and `IncidentType` enumerations (Table 7). Each requirement is met greedily by the nearest available unit of the required type (Section 7.4), which is immediately marked busy so that later searches exclude it; an ICU requirement selects the nearest hospital with a free ICU bed and reserves one bed. Exhausted resource types are recorded as unmet requirements rather than aborting the plan. Allocation is greedy per incident in arrival order, not a global optimum across simultaneous incidents.

Table 7: Allocation policy: severity requirements (left) and incident-type additions (right)

Severity	Ambulances	ICU hospital	Incident type	Fire engine	Police unit
CRITICAL	2	Yes	ROAD_ACCIDENT	Yes	Yes
HIGH	1	Yes	FIRE	Yes	Yes
MODERATE	1	No	NATURAL_DISASTER	Yes	Yes
LOW	1	No	CRIME	No	Yes
			CARDIAC_ARREST	No	No
			MEDICAL	No	No

8. JUSTIFICATION FOR DATA STRUCTURE AND ALGORITHM SELECTION

Linear scans and preprocessing. A list with linear scans spends $\Theta(n)$ per query and reuses nothing (Section 3). Here positions change far less often than they are queried — dispatch only flips availability flags, and positions change only on relocation — so an $O(n \log n)$ build is amortised well: Q queries cost $O(n \log n + Q(\log n + k))$ with the index against $\Theta(Qn)$ with the scan, and asymptotically the build is recovered after a number of queries on the order of $\log n$.

Range tree and fractional cascading. Almost every query is, or reduces to, an axis-aligned rectangle: the region query, the sector rectangles in coverage and redistribution, the bounding square of a radius query and every nearest-neighbour window. The range tree gives a worst-case, output-sensitive bound independent of the data distribution, and median splitting keeps it balanced. Cascading lowers $O(\log^2 n + k)$ to $O(\log n + k)$ for two integer arrays per node, leaving the build and space bounds unchanged; its pointers are hard to maintain under updates, but the project rebuilds on relocation, so a static structure fits. **Point location** suits the few, static, rectangular sectors: the slab method is the simplest logarithmic structure, and its $O(z^2)$ space is negligible for four sectors. **Range-based nearest neighbour** reuses the same index, touches only nearby points, and stays exact through its final pass.

Trade-offs. The index stores $O(n \log n)$ entries instead of n, relocation costs an $O(n \log n)$ rebuild instead of $O(1)$, and busy units stay in the index and are filtered out, which keeps updates cheap but makes them count towards k. Table 8 compares alternatives that were **considered conceptually, not implemented**.

Table 8: Alternative structures (conceptual comparison; only the last row is implemented)

Approach	2-D rectangle query	Nearest neighbour	Updates / space	Assessment
Linear search, brute-force distance	$\Theta(n)$	$\Theta(n)$	$O(1)$ / $O(n)$	Simplest; kept as the test oracle; does not scale
Balanced BST on x only	$O(\log n + m)$, m = points in the x-strip	Not direct	$O(\log n)$ / $O(n)$	Indexes one dimension; a strip may hold $\Theta(n)$ points
Uniform grid	Cells overlapped + their points	Nearby cells	$O(1)$ / $O(n + \text{cells})$	Good for uniform density; no worst-case bound
k-d tree [10]	$O(\sqrt{n} + k)$	Branch-and-bound	Degrades without rebalancing / $O(n)$	Lower memory, weaker range bound
R-tree [11]	No useful worst-case bound	Branch-and-bound	Dynamic / $O(n)$	Suited to disk and rectangle data; more complex
Range tree + cascading (chosen)	$O(\log n + k)$	Expanding windows	Rebuild $O(n \log n)$ / $O(n \log n)$	Strongest worst-case orthogonal bound

9. METHODOLOGY

The system follows a *build once, query many* workflow: all preprocessing happens when the city is loaded, queries are answered against the prepared structures, and only relocation rebuilds the range tree. Sections 9.1–9.3 describe the architecture, the data flow and the fifteen workflow stages.

9.1 System Architecture

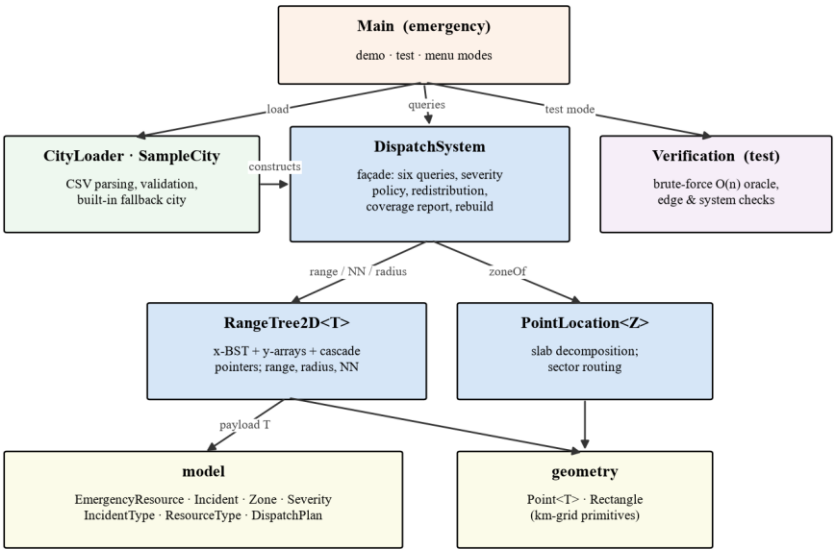


Figure 3: System architecture: packages, principal classes and dependencies

In Figure 3, `Main` selects the demo, test or menu mode. `SampleCity` and `CityLoader` construct a `DispatchSystem`, a façade that owns the resource and zone lists, a `RangeTree2D<EmergencyResource>` rebuilt whenever a unit is relocated, and a `PointLocation<Zone>` fixed once the zones are known. Both structures are generic and depend only on the `geometry` package; domain meaning enters through the payload type and the filter predicates, which is why the verification suite can test the same class as `RangeTree2D<Integer>` on random points.

9.2 Data Flow

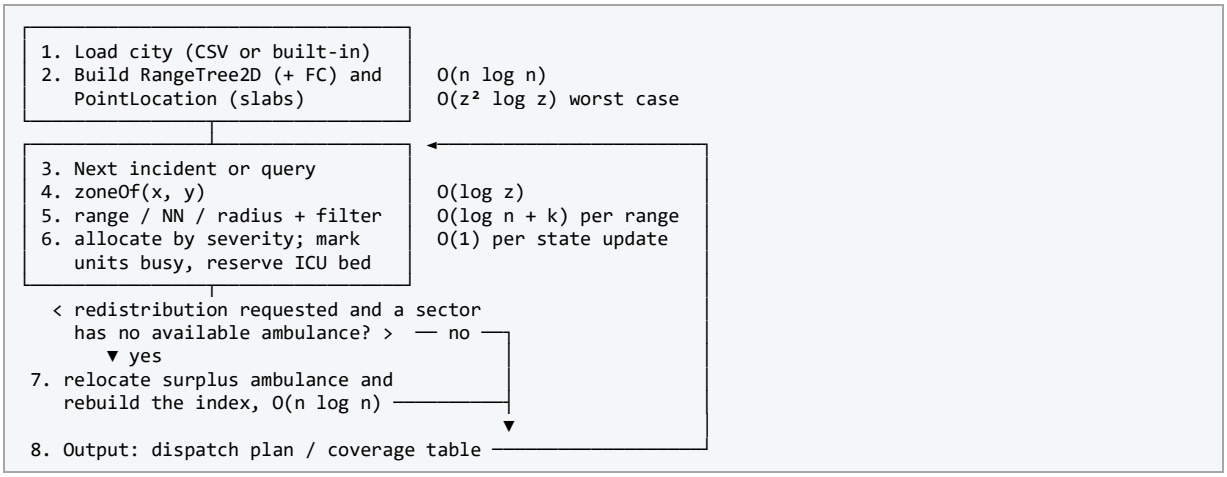
```
data/resources.csv, data/zones.csv           (fallback: SampleCity built-in dataset)
   │  CityLoader: skip comments, trim, validate
   ▼
List<EmergencyResource>, List<Zone>  ──►  new DispatchSystem(resources, zones)
   ├──► rebuildIndex(): RangeTree2D     (x-sort, build, merge-by-y, cascade pointers)
   └──► PointLocation                   (slabs of sorted y-intervals)
   ▼
Incident (x, y, type, severity)  or  query (rectangle / point / radius)
   ├──► zoneOf ──────────────────────────► owning sector
   └──► rangeQuery / nearest / withinRadius ──► candidate points ──► predicate filter
   ▼
allocate / redistributeCoverage / coverageReport ──► state updates (busy, ICU bed, relocate → rebuild)
   ▼
Output: DispatchPlan · resource lists · coverage table · redistribution log
```

9.3 Workflow

Table 9 lists the fifteen workflow stages and where each is implemented, and the flowchart below shows the control flow of a session. Redistribution is invoked explicitly (demonstration step 5, console option 6), not after every allocation.

Table 9: Workflow stages of the system

#	Stage	What happens (implemented in)
1	Input acquisition	Data read from data/; incidents from the demo script or console (Main, SampleCity)
2	CSV / data loading	Comments skipped, fields trimmed and validated, errors report file and line; built-in fallback (CityLoader)
3	Data representation	Rows become EmergencyResource or Zone objects; enumerations for types and severity (model)
4	Coordinate modelling	(x, y) in km; Point<T> with payload; inclusive Rectangle and query squares (geometry)
5	Spatial index construction	x-sort, recursive build, merge by y, cascade pointers (rebuildIndex, RangeTree2D)
6	Point-location construction	Zone rectangles converted into slabs of sorted y-intervals (PointLocation)
7	Incident input	Incident with id, location, type and severity (Incident)
8	Sector identification	Incident located among the slabs (zoneOf)
9	Spatial queries	Rectangle, radius and nearest-neighbour queries on the index (RangeTree2D)
10	Resource filtering	Type, availability, ICU and exact-distance predicates (DispatchSystem)
11	Severity-based allocation	Policy lookup; nearest available unit per requirement (allocate)
12	Resource state update	Units marked busy, ICU bed reserved, return to service (EmergencyResource)
13	Redistribution	Uncovered sectors receive the nearest surplus ambulance (redistributeCoverage)
14	Index rebuilding	Moved copy replaces the unit; range tree rebuilt (relocate, rebuildIndex)
15	Output	Dispatch plans, lists, coverage table and move log printed (DispatchPlan, coverageReport)



10. ALGORITHM DESIGN AND PSEUDOCODE

The pseudocode follows the control flow of the Java implementation. ▷ marks a comment, |A| is the length of array A, dist is Euclidean distance, SQUARE(q_x, q_y, r) is $[q_x - r, q_x + r] \times [q_y - r, q_y + r]$, *keep* is a filter predicate, REPORT-IF-INSIDE(p) reports a boundary leaf only if it satisfies all four rectangle inequalities, and LOWER-BOUND(a, y) returns the first index of a whose y-value is at least y.

10.1 Algorithm 1 — Building the Orthogonal Range Tree

Purpose. Build the x-balanced primary tree with y-sorted arrays and cascade pointers. **Input.** n points (x, y, payload). **Output.** Root of the range tree; bounding box of the points.

```
BUILD-RANGE-TREE(P)
 1  if P = ∅ then return an empty tree
 2  A ← P sorted by x;  record minX, maxX, minY, maxY      ▷ O(n log n); box used by NEAREST
 3  return BUILD(A, 0, n − 1)

BUILD(A, lo, hi)
 1  v ← new node
 2  if lo = hi then v.point ← A[lo];  v.ys ← ⟨A[lo]⟩;  return v              ▷ leaf
 3  mid ← ⌊(lo + hi) / 2⌋;  v.splitX ← A[mid].x               ▷ largest x in left subtree
 4  v.left ← BUILD(A, lo, mid);  v.right ← BUILD(A, mid + 1, hi)
 5  MERGE-BY-Y(v);  return v

MERGE-BY-Y(v)
 1  L ← v.left.ys;  R ← v.right.ys;  m ← |L| + |R|
 2  v.ys ← two-way merge of L and R by y                       ▷ ties taken from L first
 3  v.leftPtr[m] ← |L|;  v.rightPtr[m] ← |R|;  jl ← |L|;  jr ← |R|        ▷ sentinels
 4  for t ← m − 1 downto 0 do
 5      while jl > 0 and L[jl − 1].y ≥ v.ys[t].y do jl ← jl − 1
 6      while jr > 0 and R[jr − 1].y ≥ v.ys[t].y do jr ← jr − 1
 7      v.leftPtr[t] ← jl;  v.rightPtr[t] ← jr
```

Explanation. Median splitting gives height $\lceil \log_2 n \rceil$. MERGE-BY-Y is the merge step of merge sort extended with the cascade pointers; since v.ys[t].y does not increase as t decreases, jl and jr only move left, so the pointers cost O(m). **Complexity.** $T(n) = 2T(n/2) + O(n) = O(n \log n)$ time after the sort; each point is stored once per level, so the space is $O(n \log n)$.

10.2 Algorithm 2 — Orthogonal Range Search

Purpose. Report the points in $[x_1, x_2] \times [y_1, y_2]$ with one binary search per canonical node (the plain range-tree query refined by Algorithm 3). **Input.** Range tree T and a rectangle. **Output.** All stored points inside the rectangle.

```
RANGE-QUERY(T, [x1, x2] × [y1, y2])
 1  if T.root = NIL then return ∅
 2  v ← T.root
 3  while v is not a leaf and (x2 < v.splitX or x1 > v.splitX) do       ▷ find split node
 4      if x2 < v.splitX then v ← v.left else v ← v.right
 5  if v is a leaf then REPORT-IF-INSIDE(v.point);  return
 6  u ← v.left                                                  ▷ path towards x1
 7  while u is not a leaf do
 8      if x1 ≤ u.splitX then SEARCH-Y(u.right, y1, y2);  u ← u.left
 9      else u ← u.right
10  REPORT-IF-INSIDE(u.point)
11  repeat lines 6–10 symmetrically from v.right towards x2, reporting SEARCH-Y(u.left, y1, y2)
    whenever x2 ≥ u.splitX

SEARCH-Y(w, y1, y2)                                             ▷ w is a canonical node
 1  i ← LOWER-BOUND(w.ys, y1)                                   ▷ binary search, O(log n)
 2  while i < |w.ys| and w.ys[i].y ≤ y2 do report w.ys[i];  i ← i + 1
```

Explanation. A subtree is skipped only when all its x-values are provably outside $[x_1, x_2]$, and a child is canonical only when all its x-values are provably inside, so the query stays correct when several points share an x-coordinate (tested in Section 13). **Complexity.** $O(\log n)$ canonical nodes, each with an $O(\log n)$ binary search: $O(\log^2 n + k)$.

10.3 Algorithm 3 — Fractional Cascading Search

Purpose. Answer the same query with a single binary search (`RangeTree2D.rangeQuery`). **Input.** Range tree T with cascade pointers and a rectangle. **Output.** All stored points inside the rectangle.

```
RANGE-QUERY-FC(T, [x1, x2] × [y1, y2])
 1  if T.root = NIL then return ∅
 2  v ← T.root;  pos ← LOWER-BOUND(v.ys, y1)                ▷ the only binary search
 3  while v is not a leaf do                                ▷ descend to the split node
 4      if x2 < v.splitX then pos ← v.leftPtr[pos];  v ← v.left
 5      else if x1 > v.splitX then pos ← v.rightPtr[pos];  v ← v.right
 6      else break
 7  if v is a leaf then REPORT-IF-INSIDE(v.point);  return
 8  u ← v.left;  p ← v.leftPtr[pos]                         ▷ left spine
 9  while u is not a leaf do
10      if x1 ≤ u.splitX then REPORT-FROM(u.right, u.rightPtr[p], y2);  p ← u.leftPtr[p];  u ← u.left
11      else p ← u.rightPtr[p];  u ← u.right
12  REPORT-IF-INSIDE(u.point)
13  u ← v.right;  p ← v.rightPtr[pos]                       ▷ right spine
14  while u is not a leaf do
15      if x2 ≥ u.splitX then REPORT-FROM(u.left, u.leftPtr[p], y2);  p ← u.rightPtr[p];  u ← u.right
16      else p ← u.leftPtr[p];  u ← u.left
17  REPORT-IF-INSIDE(u.point)

REPORT-FROM(w, i, y2):  while i < |w.ys| and w.ys[i].y ≤ y2 do report w.ys[i];  i ← i + 1
```

Explanation. Invariant: p (or pos) is the first index in the current node's y-array with $y \geq y_1$ (Section 7.2), so every canonical node is scanned from the correct position after one array access. **Complexity.** O(log n) for the root search, O(1) per node on O(log n)-length paths and O(1) per reported point: O(log n + k).

10.4 Algorithm 4 — Point Location Using Slab Decomposition

Purpose. Identify the sector that contains an incident. **Input.** z non-overlapping labelled rectangles (build); a point (p_x, p_y) (query). **Output.** The containing zone's label, or NIL.

```
BUILD-SLABS(Z)
 1  X ← sorted distinct values of {r.minX, r.maxX : r ∈ Z}
 2  for s ← 0 to |X| − 2 do
 3      xm ← (X[s] + X[s + 1]) / 2;  C ← {r ∈ Z : r.minX ≤ xm ≤ r.maxX}
 4      sort C by minY;  store lo[s][·], hi[s][·], label[s][·]

LOCATE(px, py)
 1  if |X| < 2 or px < X[0] or px > X[|X| − 1] then return NIL
 2  s ← UPPER-BOUND(X, px) − 1;  clamp s to [0, |X| − 2]      ▷ slab by binary search on x
 3  k ← UPPER-BOUND(lo[s], py) − 1                           ▷ last interval starting ≤ py
 4  if k ≥ 0 and py ≤ hi[s][k] then return label[s][k] else return NIL
```

Explanation. UPPER-BOUND returns the first index whose value exceeds the key; clamping puts the outermost right boundary into the last slab. Intervals in a slab are disjoint and sorted, so one comparison with the candidate's upper edge decides membership. **Complexity.** Build $O(z^2 \log z)$ time and $O(z^2)$ space in the worst case; query O(log z).

10.5 Algorithm 5 — Nearest Available Resource

Purpose. Find the exact nearest point satisfying a predicate using only range queries. **Input.** Range tree T, query point q, predicate keep. **Output.** The nearest matching point, or NIL.

```
NEAREST(T, qx, qy, keep)
 1  if T.root = NIL then return NIL
 2  R ← max(qx − minX, maxX − qx, qy − minY, maxY − qy, 0);  r ← 1       ▷ r in km
 3  loop
 4      H ← {p ∈ RANGE-QUERY-FC(T, SQUARE(qx, qy, r)) : keep(p)}
 5      if H ≠ ∅ or r ≥ R then exit loop
 6      r ← min(2r, R)
 7  if H = ∅ then return NIL                                  ▷ nothing matches anywhere
 8  d ← min over p ∈ H of dist(p, q)
 9  C ← {p ∈ RANGE-QUERY-FC(T, SQUARE(qx, qy, d)) : keep(p)}  ▷ exact final pass
10  return the p ∈ C minimising dist(p, q)
```

Explanation. The window doubles until it holds a match or covers every point (half-side R); the final pass makes the answer exact (Section 7.4). Mobile units use *available ∧ type = t*, ICU hospitals *type = HOSPITAL ∧ icuBeds > 0*. **Complexity.** Each window costs O(log n + k_w), where k_w counts all points reported before filtering. With $W \le \lceil \log_2 R \rceil + 1$ windows (at most six inside the 30 × 30 km map) the total is O(W log n + Σ k_w): close to O(log n) when few points lie near q, but approaching O(W · n) when many non-matching points fall inside the windows. The project documentation states the cost as O(log n + k) per expanding window.

10.6 Algorithm 6 — Hospitals with ICU Within Radius

Purpose. Return the hospitals with a free ICU bed within distance ρ, closest first. **Input.** Range tree T, point q, radius ρ. **Output.** Hospitals sorted by distance.

```
ICU-HOSPITALS-WITHIN(T, qx, qy, ρ)
 1  B ← RANGE-QUERY-FC(T, SQUARE(qx, qy, ρ))                ▷ bounding square of the circle
 2  H ← {p ∈ B : dist(p, q) ≤ ρ and p.type = HOSPITAL and p.icuBeds > 0}
 3  sort H by dist(·, q) ascending;  return H
```

Explanation. The square limits the candidates, the distance test removes its corners and the predicate removes other types and full hospitals; the coverage report reuses the routine with 8 km. **Complexity.** O(log n + k_B) for the square, O(k_B) filtering and O(h log h) sorting of the h results.

10.7 Algorithm 7 — Severity-Based Allocation

Purpose. Build a dispatch plan according to severity and incident type. **Input.** Incident I = (x, y, type, severity). **Output.** DispatchPlan: zone, ambulances, ICU hospital, fire truck, police unit, unmet needs.

```
ALLOCATE(I)
 1  plan.zone ← LOCATE(I.x, I.y)
 2  for i ← 1 to I.severity.ambulancesNeeded do
 3      a ← NEAREST(T, I.x, I.y, available ∧ type = AMBULANCE)
 4      if a = NIL then record shortfall;  break
 5      a.available ← false;  append a to plan.ambulances
 6  if I.severity.requiresIcu then
 7      h ← NEAREST(T, I.x, I.y, type = HOSPITAL ∧ icuBeds > 0)
 8      if h ≠ NIL then h.icuBeds ← h.icuBeds − 1;  plan.hospital ← h  else record shortfall
 9  for each support type t ∈ {FIRE_TRUCK if I.type.needsFireTruck, POLICE_UNIT if I.type.needsPolice} do
10      u ← NEAREST(T, I.x, I.y, available ∧ type = t)
11      if u ≠ NIL then u.available ← false;  attach u to plan  else record shortfall
12  return plan
```

Explanation. The policy of Table 7 is read as data. Units are marked busy at once, so a CRITICAL incident gets two different ambulances and later incidents see the reduced fleet — in the demonstration P1 goes to E1, so E2 receives P2. Shortages are recorded, not treated as errors. **Complexity.** O(log z) for point location plus at most five nearest-neighbour searches (Algorithm 5); state updates are O(1).

10.8 Algorithm 8 — Ambulance Redistribution

Purpose. Restore coverage to sectors left without a free ambulance. **Input.** Current resources, zones and index. **Output.** Move log; updated positions and index.

```
REDISTRIBUTE()
 1  for each zone Z in input order do
 2      if RANGE-QUERY-FC(T, Z.area) contains an available ambulance then continue
 3      c ← centre of Z.area;  donor ← NIL;  best ← ∞
 4      for each available ambulance a in the resource list do              ▷ linear scan
 5          home ← LOCATE(a.x, a.y)
 6          if home = Z or (home ≠ NIL and |available ambulances in home.area| ≤ 1) then continue
 7          if dist(a, c) < best then donor ← a;  best ← dist(a, c)
 8      if donor = NIL then log "uncovered, no surplus ambulance";  continue
 9      replace donor by a copy located at c (same id and availability)
10      T ← BUILD-RANGE-TREE(all resources);  log the move     ▷ index rebuilt
11  return log
```

Explanation. Donors must leave at least one available ambulance in their home sector, so a move never creates a new gap; the donor nearest to the needy sector's centre is chosen by a linear scan that uses point location and a range query per candidate. **Complexity.** O(z(log n + k)) to check all zones; per uncovered zone O(n(log z + log n + k_H)) for the donor search (k_H = size of a home-zone query) plus O(n log n) for the rebuild. It is a maintenance operation invoked on demand.

11. IMPLEMENTATION

11.1 Programming Language

The system is written in Java using generics (`RangeTree2D<T>`, `PointLocation<Z>`, `Point<T>`), enumerations with fields for the dispatch policy, `java.util.function.Predicate` lambdas for query filters and `java.nio.file` with try-with-resources for input. Only the standard library is used; there are no third-party libraries or test frameworks.

11.2 Development Environment

The README states that a JDK 8 or newer is required and that a JDK 17 is installed on the development machine. `run.bat` (Windows) locates the compiler automatically, compiles all packages into `out/` and runs `emergency.Main`; `run.sh` does the same on Linux, macOS and Git Bash. An optional argument selects the mode: none (demonstration), `test` or `menu`. The program is run from the project root so that `data/*.csv` is found, and falls back to the built-in dataset otherwise. The project files do not specify an IDE or hardware.

11.3 Project Structure

The source tree src/emergency contains six packages (full tree in Appendix A.1): *geometry* (Point, Rectangle; 90 lines), *structures* (RangeTree2D, PointLocation; 421 lines), *model* (EmergencyResource, Incident, Zone, DispatchPlan, Severity, IncidentType, ResourceType; 247 lines), *system* (DispatchSystem, CityLoader, SampleCity; 475 lines), *test* (Verification;

242 lines) and the root package holding the Main driver (220 lines) — 1,695 lines of Java including comments. The spatial structures depend only on *geometry*; the domain reaches them through DispatchSystem.

11.4 Important Classes

Table 10: Important classes

Class	Principal members and role
RangeTree2D<T>	Node {splitX, ys, leftPtr, rightPtr, …}; rangeQuery, withinRadius, nearest — the spatial index (Algorithms 1, 3, 5, 6)
PointLocation<Z>	slabX, loY, hiY, label; locate — sector identification (Algorithm 4)
DispatchSystem	Façade owning resources, zones and both structures; the six queries, zoneOf, allocate, redistributeCoverage, relocate
CityLoader	dataPresent, load, loadResources, loadZones — CSV parsing and validation
EmergencyResource	Immutable id, type and position; mutable available flag and icuBeds counter
Incident, Zone, DispatchPlan	Emergency call; response sector; allocation result with zone, units and unmet needs

11.5 Data Files

`data/resources.csv` has the fields *id, name, type, x, y, available, icuBeds*, with type ∈ {AMBULANCE, FIRE_TRUCK, POLICE_UNIT, HOSPITAL} (case-insensitive); hospitals use `icuBeds` and ignore `available`, while mobile units do the reverse. `data/zones.csv` has *name, minX, minY, maxX, maxY*. Blank and #-comment lines are skipped, fields are trimmed, and a short row raises an `IOException` naming the file and line. The files are read at start-up, so the city can be edited without recompiling.

11.6 Implementation Details

Spatial indexing. Each node keeps its y-array and the two cascade pointer arrays beside the routing key:

```
private static final class Node<T> {
    Node<T> left, right;     // children (null for a leaf)
    double splitX;           // internal: max x-coordinate in the left subtree
    boolean isLeaf;
    Point<T> point;          // leaf: the single point stored here

    // Associated structure: points of this subtree sorted by ascending y.
    Point<T>[] ys;
    // Fractional-cascading pointers, length ys.length + 1 (last = sentinel).
    // leftPtr[i]  = first index j in left.ys  with left.ys[j].y  >= ys[i].y
    // rightPtr[i] = first index j in right.ys with right.ys[j].y >= ys[i].y
    int[] leftPtr;
    int[] rightPtr;
}
```

Query execution. Application questions become range queries combined with reusable predicates such as `availableType(AMBULANCE)` and `ICU_HOSPITAL`. Since the index holds every resource, filtering follows reporting: for the South-West rectangle the tree reports F1, A2, H1 and A1 (k = 4) and the predicate keeps A2 and A1. **State changes.** Availability and ICU capacity are mutable fields updated in place by dispatch, return to service and reservation, without touching the index. **Boundaries.** `Rectangle` is inclusive, so a region query would count a resource on a shared sector boundary in both sectors, whereas point location assigns it to one; no sample resource lies on a boundary. **Rebuilding.** Coordinates are final, so `relocate` replaces a unit by a moved copy with the same identity and availability

and calls `rebuildIndex`, which constructs a new `RangeTree2D` (Appendix A.3.6); the slab structure is never rebuilt.

12. COMPLEXITY ANALYSIS

Let n be the number of indexed resources, k the points reported by a range query before filtering, z the number of zones, W the number of nearest-neighbour windows, k_w the points in one window, k_B those in a radius query's bounding square, k_H those in a home-zone query and h the hospitals returned. Table 11 summarises the bounds derived in Section 10.

Table 11: Time and space complexity

Operation	Best case	Average case	Worst case	Space
Build range tree	O(n log n)	O(n log n)	O(n log n)	O(n log n)
Plain range search	O(log n)	$O(\log^2 n + k)$	$O(\log^2 n + k)$	O(k) output
Range search with fractional cascading	O(log n)	O(log n + k)	O(log n + k)	O(k) output
Build slab structure	O(z log z)	$O(z^2 \log z)$	$O(z^2 \log z)$	$O(z^2)$
Point location query	O(1)	O(log z)	O(log z)	O(1)
Nearest available unit	O(log n)	O(W log n + Σ k_w)	O(W · n)	O(max k_w)
ICU hospitals within radius	O(log n)	O(log n + k_B + h log h)	O(n + h log h)	O(k_B)
Severity allocation	O(log z + log n)	O(log z) + ≤ 5 NN searches	O(log z + W · n)	O(1) + plan
Redistribution	O(z(log n + k))	O(n(log n + k_H) + n log n) per uncovered zone	O(z · n(log n + k_H)), k_H ≤ n	O(n log n)
Mark busy / reserve ICU bed	O(1)	O(1)	O(1)	O(1)
Relocate one unit	O(n log n)	O(n log n)	O(n log n)	O(n log n)
Naïve linear scan	Θ(n)	Θ(n)	Θ(n)	O(1)

Preprocessing and memory. The build performs one O(n) merge pass per level over $\lceil \log_2 n \rceil + 1$ levels; for the sample city (n = 16) the y-arrays hold 16 × 5 = 80 references and the 15 internal nodes 2 × (64 + 15) = 158 pointer integers — a small, predictable overhead. **Query cost.** A cascaded query costs O(log n) plus a constant per reported point, whereas the scan pays Θ(n) even for an empty answer. Because k includes points later rejected by a predicate, and nearest-neighbour search runs W windows plus a final pass, Algorithms 5, 7 and 8 are stated in terms of W, k_w and k_H rather than a single O(log n + k). **Updates and scalability.** State changes are O(1); each relocation costs an O(n log n) rebuild, so redistribution is the most expensive operation and runs on demand. For queries the gap to the scan widens as n grows (Section 14.2).

13. EXPERIMENTAL SETUP AND TEST CASES

13.1 Experimental Environment

All checks are run by `emergency.test.Verification` (`run.bat test`, `./run.sh test` or `java -cp out emergency.Main test`) on a JDK 8 or newer. The randomised checks use fixed seeds (7, 11, 13), so every run is reproducible. The project records correctness outcomes only — no execution times — so correctness is evaluated empirically and performance analytically.

13.2 Dataset

The functional dataset is the sample city of Figure 1: a 30 × 30 km grid with four 15 × 15 km sectors and sixteen resources — eight ambulances (A1–A8), two fire trucks (F1, F2), two police units (P1, P2) and four hospitals (H1–H4) with 6, 3, 0 and 4 free ICU beds — all available at start-up. It is stored in data/*.csv, duplicated in SampleCity.builtIn(), and listed in full in Appendix A.8. The randomised tests generate 1–120 points (1–80 for the radius test) with integer coordinates in [0, 39], so repeated coordinates are common, together with random rectangles, query points and radii of 1–15.

13.3 Test Cases

Table 12 lists the nine checks. Each randomised check stops at its first mismatch, so a PASS means every query agreed with the brute-force reference.

Table 12: Verification test cases (RESULT: 9 passed, 0 failed)

TC	Input / scenario	Expected result	Actual result	Status
1	Rectangular range: 300 random point sets × 20 rectangles = 6,000 queries, with duplicates	Same multiset as brute-force scan	All 6,000 matched	PASS
2	Nearest neighbour: 300 sets × 10 points = 3,000 queries	Distance equals brute-force minimum ($\pm 10^{-9}$)	All 3,000 matched	PASS
3	Radius: 200 sets × 10 queries, r = 1–15 = 2,000 queries	Same set as brute-force circle test	All 2,000 matched	PASS
4	Empty structure	Size 0; empty range result; no nearest point	As expected	PASS
5	Single point (5, 5)	Found in $[0,10]^2$; not in $[6,10]^2$; nearest from (0, 0) found	As expected	PASS
6	Duplicates: four points at (5, 5), one at (9, 9); query [5,5] × [5,5]	4 points reported	4 reported	PASS
7	Point location of (5, 5), (20, 20), (100, 100)	South-West, North-East, none	As expected	PASS
8	Allocation: CRITICAL road accident at (19, 24)	Available ambulances drop by exactly 2	Dropped by 2	PASS
9	Redistribution after MODERATE calls at (20, 6) and (26, 9)	South-East emptied, then covered again	Covered again	PASS

14. RESULTS AND DISCUSSION

14.1 Functional Results

Tables 13 and 14 reproduce the output of the scripted demonstration (`java emergency.Main`), whose values also appear in the project presentation; distances are straight-line kilometres.

Table 13: Functional results of the demonstration run

Operation	Input	Output of the demonstration
1. Region	South-West sector [0, 15] × [0, 15]	A2 (6, 12) and A1 (3, 4), both available
2. Nearest ambulance	Accident at (18, 23)	A5 at (22, 25), 4.47 km
3. ICU within 5 km	Centre (7, 11)	City-General (H1), 3.16 km, 6 ICU beds
4. Allocation E1	ROAD_ACCIDENT, CRITICAL, (19, 24)	North-East; A5 (3.16 km), A4 (6.71 km); ICU H2 Northside-Med (7.07 km); fire F2 (5.39 km); police P1 (12.53 km)
4. Allocation E2	FIRE, HIGH, (5, 6)	South-West; A1 (2.83 km); ICU H1 (4.12 km); fire F1 (2.00 km); police P2 (18.68 km)
4. Allocation E3	MEDICAL, MODERATE, (24, 10)	South-East; A8 (2.24 km)
5. Redistribution	Fresh city; calls at (20, 6) and (26, 9) consume A7, A8	South-East: A6 relocated from (27, 20) to (22.5, 7.5)

Table 14: Coverage report (operation 6) before and after redistribution

Zone	Ambulances before	ICU < 8 km	Nearest to centre before	Ambulances after	Nearest to centre after
North-West	2	yes	A3 @ 2.55 km	2	A3 @ 2.55 km
North-East	2	NO	A5 @ 2.55 km	1	A5 @ 2.55 km
South-West	2	yes	A2 @ 4.74 km	2	A2 @ 4.74 km
South-East	0 (UNDER-COVERED)	yes	A6 @ 13.29 km	1	A6 @ 0.00 km

The results also exercise the filters and state handling. E1's ICU hospital is Northside-Med at 7.07 km because East-Care (H3) has no free ICU bed; for the same reason North-East shows no ICU within 8 km of its centre, although East-Care is only 5.15 km away. E2 receives P2 at 18.68 km because the nearer P1 was committed to E1 — correct greedy behaviour that also shows the cost of serving incidents one at a time. A6 is the redistribution donor as the nearest ambulance in a sector holding two (13.29 km from the South-East centre, against 17.10 km for A2 and 17.51 km for A5), and the South-East response distance falls from 13.29 km to 0.00 km while North-East keeps A5.

14.2 Complexity Results

Rectangular search drops from Θ(n) for the naïve scan to O(log n + k) with fractional cascading, with O(log² n + k) for the plain range tree in between. Figure 4 plots the dominant terms (the common output term k omitted), computed from the formulas as in the project presentation's chart; they are not measured running times.

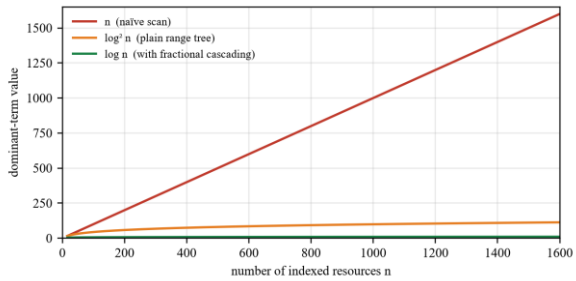


Figure 4: Analytical growth of the dominant query term: n, log² n and log n (computed from the formulas; not measured running times)

At n = 1,600 the terms are 1,600, about 113 and about 10.6. Doubling n doubles the scan but adds only one to $\log_2 n$, so the advantage grows with the fleet, while for the sixteen-resource sample city the difference is small in absolute terms.

14.3 Correctness Results

All nine checks passed with zero failures. The 11,000 randomised queries — 6,000 range, 3,000 nearest-neighbour and 2,000 radius — matched independent brute-force results exactly, including point sets with many repeated coordinates, and the edge cases and system behaviours met their specifications. With the hand-traced examples of Sections 7 and 14.1 reproducing the demonstration output, this gives good confidence in the cascade pointers, the duplicate-safe routing and the exact nearest-neighbour pass.

14.4 Discussion

Preprocessing pays off because the workload is query-heavy: the tree is built once, while allocation, radius search and coverage reporting issue many range queries that never modify it. Fractional cascading works because all canonical nodes search for the same y_1 in nested subsequences, so one stored answer per entry replaces O(log n) repeated binary searches — an idea applied to range trees by Lueker [5] and Willard [6] — without compromising exactness, and each query touches only the part of the city near its region.

The model has clear limitations: straight-line rather than road distances; a full O(n log n) rebuild on every relocation; busy units and other types inflating k; greedy per-incident allocation; a linear donor scan; and $O(z^2)$ worst-case slab space. The evaluation covers correctness only — sixteen resources, random sets of at most 120 points and no recorded timings. The presentation describes the demonstration answers as sub-second, but without measurements this report makes no quantitative runtime claim; its performance conclusions rest on the complexity analysis.

15. CONCLUSION AND FUTURE SCOPE

15.1 Conclusion

The project addressed the cost of answering repeated dispatch queries by scanning every resource. It formulated the task as orthogonal range reporting, nearest-neighbour search, filtered radius search and planar point location over resources with mutable state, and implemented in Java an orthogonal range tree with fractional cascading and a slab-decomposition point-location structure, with severity-based allocation, coverage reporting and redistribution on top. The index is built in O(n log n) time and space, reports rectangles in $O(\log n + k)$ instead of $\Theta(n)$, and locates sectors in O(log z). All nine checks, including 11,000 randomised brute-force comparisons, passed, and the sample-city demonstration behaved as expected, notably restoring coverage to the South-East sector. Classical computational-geometry structures thus turn a per-query linear scan into an exact, output-sensitive logarithmic query, at the cost of extra storage and a rebuild when resources move.

15.2 Future Scope

The project documentation identifies three directions:

1. **Road-network travel time** in place of straight-line distance.

2. **Incremental updates** in place of rebuilding the whole index after a relocation.
3. **Predictive pre-positioning** of units before calls arrive.

`docs/REPORT.md` also mentions a trapezoidal map or persistent search trees [8] for O(n)-space point location, real-time traffic weighting, batching of simultaneous incidents and a map-based GUI. Proposed here as further enhancements (not present in the system): measured benchmarks against the linear scan at increasing n, and separate indexes for available units of each type so that k counts only usable units.

REFERENCES

Project materials (primary sources for all project-specific information)

[P1] README.md — Project 29: Intelligent Emergency Resource Allocation & Ambulance Dispatch System.

[P2] docs/REPORT.md — project write-up: theory, pseudocode, complexity and results.

[P3] Emergency_Dispatch_System.pptx — project presentation (13 slides).

[P4] DSA_Presentation_Emergency_Dispatch.pptx — project presentation (5 slides).

[P5] Java source code, src/emergency/ (Main, geometry, structures, model, system and test packages).

[P6] Data files data/resources.csv and data/zones.csv; build scripts run.bat and run.sh.

External academic and technical references

[1] M. de Berg, O. Cheong, M. van Kreveld and M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed. Berlin: Springer, 2008 (Ch. 5, Orthogonal Range Searching; Ch. 6, Point Location).

[2] B. Chazelle and L. J. Guibas, "Fractional cascading: I. A data structuring technique," *Algorithmica*, vol. 1, pp. 133–162, 1986.

[3] J. L. Bentley, "Decomposable searching problems," *Information Processing Letters*, vol. 8, no. 5, pp. 244–251, 1979.

[4] J. L. Bentley, "Multidimensional divide-and-conquer," *Communications of the ACM*, vol. 23, no. 4, pp. 214–229, 1980.

[5] G. S. Lueker, "A data structure for orthogonal range queries," in *Proc. 19th Annual Symposium on Foundations of Computer Science*, 1978, pp. 28–34.

[6] D. E. Willard, "New data structures for orthogonal range queries," *SIAM Journal on Computing*, vol. 14, no. 1, pp. 232–253, 1985.

[7] D. P. Dobkin and R. J. Lipton, "Multidimensional searching problems," *SIAM Journal on Computing*, vol. 5, no. 2, pp. 181–186, 1976.

[8] N. Sarnak and R. E. Tarjan, "Planar point location using persistent search trees," *Communications of the ACM*, vol. 29, no. 7, pp. 669–679, 1986.

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[10] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.

[11] A. Guttman, "R-trees: A dynamic index structure for spatial searching," in *Proc. ACM SIGMOD International Conference on Management of Data*, 1984, pp. 47–57.

[12] F. P. Preparata and M. I. Shamos, *Computational Geometry: An Introduction*. New York: Springer-Verlag, 1985.

[13] Oracle, *Java Platform, Standard Edition 8 API Specification*. https://docs.oracle.com/javase/8/docs/api/

[14] Oracle, *Java Platform, Standard Edition 17 API Specification*. https://docs.oracle.com/en/java/javase/17/docs/api/

APPENDIX

A.1 Project Directory Structure

```
DSA PROJECT/
├── README.md                  project overview, build instructions, test summary
├── run.bat / run.sh           one-command build + run (auto-detects the JDK)
├── data/
│   ├── resources.csv          ambulances, fire/police units, hospitals
│   └── zones.csv              city sectors for point location
├── docs/
│   └── REPORT.md              project write-up
└── src/emergency/
    ├── Main.java              driver: demo / test / menu
    ├── geometry/              Point, Rectangle
    ├── structures/            RangeTree2D (+ fractional cascading), PointLocation
    ├── model/                 ResourceType, Severity, IncidentType, EmergencyResource,
    │                          Incident, Zone, DispatchPlan
    ├── system/                DispatchSystem, CityLoader, SampleCity
    └── test/                  Verification (brute-force correctness suite)
```

A.2 Build and Run Commands

```
run.bat                 REM Windows: scripted demonstration of all queries
run.bat test            REM correctness suite (vs brute force)
run.bat menu            REM interactive console
./run.sh [test|menu]    # Linux / macOS / Git Bash

javac -d out <all .java files under src>      # manual build
java -cp out emergency.Main [test|menu]        # manual run (from the project root)

Interactive console:  1) ambulances in region   2) nearest ambulance   3) ICU hospitals in
radius
                      4) dispatch incident: x y severity(C/H/M/L)
type(ACC/FIRE/CARD/MED/DIS/CRIME)
                      5) coverage map          6) redistribute         0) quit
```

A.3 Selected Source Code

The following excerpts are copied verbatim from the project source files.

A.3.1 RangeTree2D — merge by y with cascade pointers

```
/** Merges the children's y-arrays and computes fractional-cascading pointers. */
private void mergeByY(Node<T> v) {
    Point<T>[] L = v.left.ys;
    Point<T>[] R = v.right.ys;
    int n = L.length + R.length;
    Point<T>[] merged = newArray(n);

    int i = 0, j = 0, k = 0;
    while (i < L.length && j < R.length) {
        merged[k++] = (L[i].y() <= R[j].y()) ? L[i++] : R[j++];
    }
    while (i < L.length) merged[k++] = L[i++];
    while (j < R.length) merged[k++] = R[j++];
    v.ys = merged;

    // Cascade pointers (sentinel slot at index n points past the children).
    int[] lp = new int[n + 1];
    int[] rp = new int[n + 1];
    lp[n] = L.length;
    rp[n] = R.length;
    int jl = L.length, jr = R.length;
    for (int t = n - 1; t >= 0; t--) {
        double yv = merged[t].y();
        while (jl - 1 >= 0 && L[jl - 1].y() >= yv) jl--;
        lp[t] = jl;
```

```
        while (jr - 1 >= 0 && R[jr - 1].y() >= yv) jr--;
        rp[t] = jr;
    }
    v.leftPtr = lp;
    v.rightPtr = rp;
}
```

A.3.2 RangeTree2D — range query with fractional cascading

```
/** Reports every stored point whose coordinates fall inside {@code rect}. */
public List<Point<T>> rangeQuery(Rectangle rect) {
    List<Point<T>> out = new ArrayList<Point<T>>();
    if (root == null) return out;

    final double x1 = rect.minX(), x2 = rect.maxX();
    final double y1 = rect.minY(), y2 = rect.maxY();

    // Descend to the split node while maintaining pos = first index >= y1.
    Node<T> v = root;
    int pos = lowerBoundY(root.ys, y1);
    while (!v.isLeaf) {
        if (x2 < v.splitX) {              // both bounds go left
            pos = v.leftPtr[pos];
            v = v.left;
        } else if (x1 > v.splitX) {       // both bounds go right
            pos = v.rightPtr[pos];
            v = v.right;
        } else {
            break;                         // split node found
        }
    }

    if (v.isLeaf) {                        // whole query collapsed to one leaf
        addLeafIfInside(v.point, x1, x2, y1, y2, out);
        return out;
    }

    final Node<T> split = v;
    final int posSplit = pos;

    // ---- left spine: follow x1, report right children as canonical ----
    Node<T> lv = split.left;
    int lpos = split.leftPtr[posSplit];
    while (!lv.isLeaf) {
        if (x1 <= lv.splitX) {
            report(lv.right, lv.rightPtr[lpos], y2, out);
            lpos = lv.leftPtr[lpos];
            lv = lv.left;
        } else {
            lpos = lv.rightPtr[lpos];
            lv = lv.right;
        }
    }
    addLeafIfInside(lv.point, x1, x2, y1, y2, out);

    // ---- right spine: follow x2, report left children as canonical ----
    Node<T> rv = split.right;
    int rpos = split.rightPtr[posSplit];
    while (!rv.isLeaf) {
        if (x2 >= rv.splitX) {
            report(rv.left, rv.leftPtr[rpos], y2, out);
            rpos = rv.rightPtr[rpos];
            rv = rv.right;
        } else {
            rpos = rv.leftPtr[rpos];
            rv = rv.left;
        }
    }
    addLeafIfInside(rv.point, x1, x2, y1, y2, out);

    return out;
}

/** Reports a canonical subtree: its y-array from {@code start} while y <= y2. */
```

```
private void report(Node<T> w, int start, double y2, List<Point<T>> out) {
    Point<T>[] ys = w.ys;
    for (int p = start; p < ys.length && ys[p].y() <= y2; p++) {
        out.add(ys[p]);
    }
}

private void addLeafIfInside(Point<T> p, double x1, double x2,
                             double y1, double y2, List<Point<T>> out) {
    if (p.x() >= x1 && p.x() <= x2 && p.y() >= y1 && p.y() <= y2) {
        out.add(p);
    }
}

/** First index i with {@code ys[i].y >= key}; returns ys.length if none. */
private int lowerBoundY(Point<T>[] ys, double key) {
    int lo = 0, hi = ys.length;           // search in [lo, hi)
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (ys[mid].y() >= key) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

A.3.3 RangeTree2D — exact nearest neighbour by expanding windows

```
public Point<T> nearest(double qx, double qy, Predicate<Point<T>> keep) {
    if (root == null) return null;

    // Largest half-side ever needed to cover the whole data set.
    double cover = Math.max(Math.max(qx - minX, maxX - qx),
                            Math.max(qy - minY, maxY - qy));
    if (cover < 0) cover = 0;

    double r = 1.0;                        // start with a 1 km window
    List<Point<T>> hits = null;
    while (true) {
        hits = rangeQuery(Rectangle.around(qx, qy, r), keep);
        if (!hits.isEmpty() || r >= cover) break;
        r = Math.min(r * 2.0, cover);
    }
    if (hits.isEmpty()) return null;       // nothing matches anywhere

    double best = Double.POSITIVE_INFINITY;
    for (Point<T> p : hits) best = Math.min(best, p.distanceTo(qx, qy));

    // Exact pass: the true nearest lies within distance 'best'.
    Point<T> winner = null;
    double winnerDist = Double.POSITIVE_INFINITY;
    for (Point<T> p : rangeQuery(Rectangle.around(qx, qy, best), keep)) {
        double d = p.distanceTo(qx, qy);
        if (d < winnerDist) { winnerDist = d; winner = p; }
    }
    return winner;
}
```

A.3.4 PointLocation — slab construction and query

```
public PointLocation(List<Rectangle> rects, List<Z> labels) {
    if (rects.size() != labels.size()) {
        throw new IllegalArgumentException("rects and labels differ in size");
    }

    // 1. Distinct, sorted x-boundaries -> slabs.
    TreeSet<Double> xs = new TreeSet<Double>();
    for (Rectangle r : rects) { xs.add(r.minX()); xs.add(r.maxX()); }
    slabX = new double[xs.size()];
    int idx = 0;
    for (Double d : xs) slabX[idx++] = d;

    int slabs = Math.max(slabX.length - 1, 0);
    loY = new double[slabs][];
```

```
    hiY = new double[slabs][];
    label = new Object[slabs][];

    // 2. For each slab, gather the rectangles covering it, sorted by lower y.
    for (int s = 0; s < slabs; s++) {
        double mid = (slabX[s] + slabX[s + 1]) / 2.0;
        List<Integer> cover = new ArrayList<Integer>();
        for (int i = 0; i < rects.size(); i++) {
            Rectangle r = rects.get(i);
            if (r.minX() <= mid && mid <= r.maxX()) cover.add(i);
        }
        cover.sort((a, b) -> Double.compare(rects.get(a).minY(),
                                            rects.get(b).minY()));
        int m = cover.size();
        loY[s] = new double[m];
        hiY[s] = new double[m];
        label[s] = new Object[m];
        for (int k = 0; k < m; k++) {
            Rectangle r = rects.get(cover.get(k));
            loY[s][k] = r.minY();
            hiY[s][k] = r.maxY();
            label[s][k] = labels.get(cover.get(k));
        }
    }
}

/** Returns the region containing (px, py), or {@code null} if outside all. */
@SuppressWarnings("unchecked")
public Z locate(double px, double py) {
    if (slabX.length < 2) return null;
    if (px < slabX[0] || px > slabX[slabX.length - 1]) return null;

    // Slab index: largest s with slabX[s] <= px (clamped to last slab).
    int s = upperBound(slabX, px) - 1;
    if (s < 0) s = 0;
    if (s >= loY.length) s = loY.length - 1;

    // Within the slab, find the disjoint y-interval containing py.
    double[] lo = loY[s];
    double[] hi = hiY[s];
    int k = upperBound(lo, py) - 1;        // last interval with lo <= py
    if (k >= 0 && py <= hi[k]) {
        return (Z) label[s][k];
    }
    return null;
}

/** First index i with {@code a[i] > key}. */
private static int upperBound(double[] a, double key) {
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] > key) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

A.3.5 DispatchSystem — severity-based allocation

```
public DispatchPlan allocate(Incident inc) {
    DispatchPlan plan = new DispatchPlan(inc);
    plan.setZone(zoneOf(inc.x(), inc.y()));

    int need = inc.severity().ambulancesNeeded();
    for (int i = 0; i < need; i++) {
        EmergencyResource amb = nearestAvailable(inc.x(), inc.y(),
                ResourceType.AMBULANCE);
        if (amb == null) {
            plan.addUnmet("No ambulance available (" + (need - i) + " short)");
            break;
        }
        amb.setAvailable(false);            // taken off the board
        plan.addAmbulance(amb);
```

```
    }

    if (inc.severity().requiresIcu()) {
        EmergencyResource h = nearestIcuHospital(inc.x(), inc.y());
        if (h != null) { h.reserveIcuBed(); plan.setHospital(h); }
        else plan.addUnmet("No ICU hospital available");
    }

    if (inc.type().needsFireTruck()) {
        EmergencyResource f = nearestAvailable(inc.x(), inc.y(),
                ResourceType.FIRE_TRUCK);
        if (f != null) { f.setAvailable(false); plan.setFireTruck(f); }
        else plan.addUnmet("No fire truck available");
    }

    if (inc.type().needsPolice()) {
        EmergencyResource pol = nearestAvailable(inc.x(), inc.y(),
                ResourceType.POLICE_UNIT);
        if (pol != null) { pol.setAvailable(false); plan.setPolice(pol); }
        else plan.addUnmet("No police unit available");
    }

    return plan;
}

private EmergencyResource nearestIcuHospital(double x, double y) {
    Point<EmergencyResource> p = tree.nearest(x, y, ICU_HOSPITAL);
    return p == null ? null : p.payload();
}
```

A.3.6 DispatchSystem — redistribution, relocation and index rebuild

```
public List<String> redistributeCoverage() {
    List<String> log = new ArrayList<String>();
    for (Zone z : zones) {
        if (!availableAmbulancesInRegion(z.area()).isEmpty()) continue;

        double cx = (z.area().minX() + z.area().maxX()) / 2.0;
        double cy = (z.area().minY() + z.area().maxY()) / 2.0;

        EmergencyResource donor = nearestSurplusAmbulance(z);
        if (donor == null) {
            log.add(z.name() + ": uncovered, no surplus ambulance to move");
            continue;
        }
        log.add(String.format("%s: relocating %s from (%.1f,%.1f) to (%.1f,%.1f)",
                z.name(), donor.id(), donor.x(), donor.y(), cx, cy));
        relocate(donor.id(), cx, cy);
    }
    return log;
}

/** Nearest available ambulance that sits in a zone holding more than one. */
private EmergencyResource nearestSurplusAmbulance(Zone needy) {
    double cx = (needy.area().minX() + needy.area().maxX()) / 2.0;
    double cy = (needy.area().minY() + needy.area().maxY()) / 2.0;
    EmergencyResource best = null;
    double bestD = Double.POSITIVE_INFINITY;
    for (EmergencyResource r : resources) {
        if (r.type() != ResourceType.AMBULANCE || !r.isAvailable()) continue;
        Zone home = zoneOf(r.x(), r.y());
        if (home == needy) continue;
        if (home != null && availableAmbulancesInRegion(home.area()).size() <= 1) {
            continue;   // don't strip a zone down to zero
        }
        double d = dist(r, cx, cy);
        if (d < bestD) { bestD = d; best = r; }
    }
    return best;
}

/** Moves a resource to a new position and rebuilds the spatial index. */
public void relocate(String resourceId, double newX, double newY) {
    for (int i = 0; i < resources.size(); i++) {
```

```
        EmergencyResource r = resources.get(i);
        if (r.id().equals(resourceId)) {
            EmergencyResource moved = new EmergencyResource(
                    r.id(), r.name(), r.type(), newX, newY);
            moved.setAvailable(r.isAvailable());
            resources.set(i, moved);
            rebuildIndex();
            return;
        }
    }
}

/** (Re)builds the range tree from the current resource positions. */
private void rebuildIndex() {
    List<Point<EmergencyResource>> pts =
            new ArrayList<Point<EmergencyResource>>();
    for (EmergencyResource r : resources) {
        pts.add(new Point<EmergencyResource>(r.x(), r.y(), r));
    }
    this.tree = new RangeTree2D<EmergencyResource>(pts);
}
```

A.3.7 Verification — range query against brute force

```
private void rangeQueriesMatchBruteForce() {
    Random rng = new Random(7);
    boolean ok = true;
    for (int t = 0; t < 300 && ok; t++) {
        List<Point<Integer>> pts = randomPoints(rng, 1 + rng.nextInt(120), 40);
        RangeTree2D<Integer> tree = new RangeTree2D<Integer>(pts);
        for (int q = 0; q < 20; q++) {
            Rectangle rect = randomRect(rng, 40);
            List<String> got = keys(tree.rangeQuery(rect));
            List<String> exp = bruteRange(pts, rect);
            Collections.sort(got);
            Collections.sort(exp);
            if (!got.equals(exp)) { ok = false; break; }
        }
    }
    check("range query == brute force (6000 queries, with duplicates)", ok);
}
```

A.4 Complete Demonstration Output

```
Using JDK at: C:\Users\SIVAGURU R.M\AppData\Local\Programs\Microsoft\jdk-17.0.10.7-hotspot\bin\
============================================================
 INTELLIGENT EMERGENCY RESOURCE ALLOCATION & DISPATCH SYSTEM
============================================================
Data source : data/*.csv
City model  : 30 km x 30 km, 4 sectors, 16 resources in a 2-D range tree with fractional cascading.
Distances are straight-line kilometres.


------------------------------------------------------------
1) Available ambulances inside a region  (orthogonal range search)
------------------------------------------------------------
   Region [0.00..15.00] x [0.00..15.00] (South-West):
      A2    Ambulance-2        AMBULANCE   @ (6.0, 12.0) [available]
      A1    Ambulance-1        AMBULANCE   @ (3.0, 4.0) [available]

------------------------------------------------------------
2) Nearest available ambulance to an accident  (nearest neighbour)
------------------------------------------------------------
   Accident at (18, 23):
      nearest -> A5    Ambulance-5        AMBULANCE   @ (22.0, 25.0) [available]   (4.47 km)

------------------------------------------------------------
3) Hospitals with ICU within 5 km  (range search + filter)
------------------------------------------------------------
   Searching around (7, 11), radius 5 km:
      City-General  (3.16 km, 6 ICU beds)

------------------------------------------------------------
4) Severity-based allocation  (+ point location for the sector)
```

```
------------------------------------------------------------
Incident E1: ROAD_ACCIDENT (Life-threatening) at (19.0, 24.0)
   Zone: North-East
   Dispatched:
      - Ambulance       A5    Ambulance-5        (3.16 km)
      - Ambulance       A4    Ambulance-4        (6.71 km)
      - Hospital (ICU)  H2    Northside-Med      (7.07 km)
      - Fire truck      F2    Fire-Engine-2      (5.39 km)
      - Police unit     P1    Patrol-1           (12.53 km)
Incident E2: FIRE (Serious) at (5.0, 6.0)
   Zone: South-West
   Dispatched:
      - Ambulance       A1    Ambulance-1        (2.83 km)
      - Hospital (ICU)  H1    City-General       (4.12 km)
      - Fire truck      F1    Fire-Engine-1      (2.00 km)
      - Police unit     P2    Patrol-2           (18.68 km)
Incident E3: MEDICAL (Urgent) at (24.0, 10.0)
   Zone: South-East
   Dispatched:
      - Ambulance       A8    Ambulance-8        (2.24 km)

------------------------------------------------------------
5) Dynamic redistribution after dispatch
------------------------------------------------------------
   Two calls in South-East have just consumed A7 and A8.
   Coverage BEFORE redistribution:
ZONE             AMBUL.   ICU<8km    RESPONSE (centre)
   ------------------------------------------------------------
   North-West    2        yes        A3 @ 2.55 km
   North-East    2        NO         A5 @ 2.55 km
   South-West    2        yes        A2 @ 4.74 km
   South-East    0        yes        A6 @ 13.29 km       <-- UNDER-COVERED
   Redistribution actions:
      South-East: relocating A6 from (27.0,20.0) to (22.5,7.5)
   Coverage AFTER redistribution:
ZONE             AMBUL.   ICU<8km    RESPONSE (centre)
   ------------------------------------------------------------
   North-West    2        yes        A3 @ 2.55 km
   North-East    1        NO         A5 @ 2.55 km
   South-West    2        yes        A2 @ 4.74 km
   South-East    1        yes        A6 @ 0.00 km

------------------------------------------------------------
6) Coverage map for planning
------------------------------------------------------------
   The per-sector table above is the coverage map. Sectors
   flagged UNDER-COVERED have no free ambulance and are first
   in line for additional units or relocation.

(Run "java emergency.Main test" for the correctness suite.)
```

A.5 Complete Verification Output

```
Using JDK at: C:\Users\SIVAGURU R.M\AppData\Local\Programs\Microsoft\jdk-17.0.10.7-hotspot\bin\
============================================================
 SELF-TEST SUITE
============================================================
   [PASS] range query == brute force (6000 queries, with duplicates)
   [PASS] nearest neighbour == brute force (3000 queries)
   [PASS] radius query == brute force circle (2000 queries)
   [PASS] empty tree behaves (no points, null nearest)
   [PASS] single-point tree (inside / outside / nearest)
   [PASS] duplicate coordinates all reported (expected 4)
   [PASS] point location routes incidents to the right sector
   [PASS] severity allocation removes the right number of ambulances
   [PASS] redistribution restores coverage to an emptied sector

   RESULT: 9 passed, 0 failed
   ALL CHECKS PASSED
```

A.6 Additional Pseudocode

```
COVERAGE-REPORT(ρ)                                  ▷ ρ = ICU radius (8 km in the demo)
 1  for each zone Z do
 2      c ← centre of Z.area
 3      a ← |{p ∈ RANGE-QUERY-FC(T, Z.area) : p is an available ambulance}|
 4      icu ← ICU-HOSPITALS-WITHIN(T, c.x, c.y, ρ) ≠ ∅
 5      u ← NEAREST(T, c.x, c.y, available ∧ type = AMBULANCE)
 6      print Z.name, a, icu, u and dist(u, c);  flag "UNDER-COVERED" if a = 0

LOAD-CITY(dir)
 1  if resources.csv and zones.csv are readable in dir then
 2      for each line not blank and not starting with '#' do
 3          split on ',', trim fields; if too few fields then raise error(file, line)
 4          create EmergencyResource (hospital: use icuBeds; mobile: use available) or Zone
 5      return new DispatchSystem(resources, zones)          ▷ builds both structures
 6  else return SampleCity.builtIn()                          ▷ identical built-in dataset

LOWER-BOUND(ys, key)                                ▷ first index with ys[i].y ≥ key
 1  lo ← 0;  hi ← |ys|
 2  while lo < hi do
 3      mid ← ⌊(lo + hi) / 2⌋
 4      if ys[mid].y ≥ key then hi ← mid else lo ← mid + 1
 5  return lo
```

A.7 Additional Test Cases from the Demonstration

The following scenarios were traced by hand from the algorithms and compared with the demonstration output of Appendix A.4.

Table 15: Additional functional test cases traced from the algorithms

Scenario	Hand-traced expectation	Observed	Status
Region $[0,15]^2$ over all 16 resources	Index reports F1, A2, H1, A1; ambulance filter keeps A2, A1	A2, A1	Match
Nearest ambulance to (18, 23)	Windows r = 1, 2 empty; r = 4 finds A5; d = $\sqrt{20} \approx 4.47$; final pass returns A5	A5, 4.47 km	Match
ICU within 5 km of (7, 11)	Square [2,12] × [6,16] reports F1, A2, H1; filters keep H1 ($\sqrt{10} \approx 3.16$ km)	City-General, 3.16 km	Match
Point location of E1 (19, 24)	Slab 1 ([15, 30]); interval [15, 30] → North-East	North-East	Match
Police unit for E2 after E1	P1 busy after E1, so nearest available is P2 at $\sqrt{349} \approx 18.68$ km	P2, 18.68 km	Match
North-East ICU < 8 km flag	H3 at 5.15 km has 0 ICU beds; H2 at 10.8 km is outside the radius	NO	Match
Redistribution donor	SE centre (22.5, 7.5): A6 13.29 km, A2 17.10 km, A5 17.51 km; A6 chosen	A6 relocated	Match

A.8 Sample-City Dataset

Table 16: Sample-city resources (AMB ambulance, FIRE fire truck, POL police unit, HOSP hospital)

ID	Type	(x, y) km	ICU	ID	Type	(x, y) km	ICU	ID	Type	(x, y) km	ICU
A1	AMB	(3, 4)	—	A7	AMB	(20, 6)	—	H1 City-General	HOSP	(4, 10)	6
A2	AMB	(6, 12)	—	A8	AMB	(26, 9)	—	H2 Northside-Med	HOSP	(12, 25)	3
A3	AMB	(10, 22)	—	F1	FIRE	(5, 8)	—	H3 East-Care	HOSP	(25, 18)	0
A4	AMB	(13, 27)	—	F2	FIRE	(24, 22)	—	H4 Metro-Trauma	HOSP	(21, 7)	4
A5	AMB	(22, 25)	—	P1	POL	(8, 18)	—				
A6	AMB	(27, 20)	—	P2	POL	(23, 11)	—				